Advanced Computer Architecture 1st Term, 2026-2027 Integrating Project

Smith-Waterman Algorithm: Full Alignment Reconstruction with Local Alignment Comparison between CUDA and C

Group 3

Alcasid, Aizy
Bilanes, Kendrick
Corpuz, Gerald
Roa, Luis

I. Abstract

Finding matching patterns in data is essential for tasks like DNA research, plagiarism detection, and malware analysis. The Smith-Waterman algorithm is a highly accurate method for finding these local sequence alignments, but it is slow for large datasets because of its quadratic time complexity. Using shortcuts reduces accuracy, meaning the full workload must be processed to guarantee maximum sensitivity. This project speeds up the algorithm by using GPU parallelization to handle its most demanding workload: the matrix-filling step. While a standard CPU implementation computes this dynamic programming scoring matrix one cell at a time sequentially, our CUDA approach groups independent cells into anti-diagonals. This structure allows multiple GPU threads to calculate matrix cells concurrently without breaking the algorithm's data dependencies. The parallel CUDA implementation will then be tested against a sequential C baseline program to measure the execution time speedup. Ultimately, this project will demonstrate how data-level parallelization can efficiently process heavy computational workloads while ensuring the exact and accurate reconstruction of optimal local alignments.

Index Terms: Smith-Waterman Algorithm, Local Sequence Alignment, CUDA Parallelization, Dynamic Programming

II. Description

Various fields require the ability to find optimal local sequence alignments to identify commonalities between different data sets. Examples include plagiarism detection by finding matching word segments in sentences or notes in songs, research by matching DNA, RNA, or protein sequences, and identifying cybersecurity attack patterns by comparing malware binaries.

The Smith-Waterman algorithm is a robust dynamic programming solution tailored for this exact problem. It is designed to find the best matching subsequence pair between two sequences by constructing a scoring matrix. However, there is a critical need to parallelize this algorithm. Smith-Waterman exhibits quadratic time complexity, making it computationally heavy for large datasets. While shortcuts exist to speed up the process, they make the algorithm inaccurate because full workload processing is required to guarantee maximum sensitivity and find the exact optimal alignment. With that, data-level parallelization becomes essential in implementing the project.

Implementing the project would require two sequences being compared and their corresponding scoring parameters, which dictate values for matches, mismatches, insertions, or deletions. An SIMT paradigm will be used to offload the computationally demanding matrix-filling step, which creates the dynamic programming scoring matrix, to the GPU. Standard CPU implementations construct the scoring matrix sequentially row-by-row. The

CUDA implementation will compute multiple independent matrix cells concurrently to achieve a speedup. For the parallelization process, because each cell depends on its left, upper, and upper-left neighbours, the matrix cannot be computed all at once. To solve this problem, the matrix will be further divided into anti-diagonals. CUDA threads will concurrently calculate the independent cells within the same anti-diagonal, using a synchronized kernel launch to ensure completion before advancing to the next. This will all be further discussed in the following sections.

III. Proposed Algorithm

The Smith-Waterman algorithm was developed by Temple Smith and Michael Waterman and published in 1981 [1]. Originally developed for finding common molecular subsequences in DNA, RNA, or protein [1], the Smith-Waterman algorithm is a popular dynamic programming algorithm for finding the local sequence alignment (i.e., the two sequences being compared do not have to be an exact match; rather, they just have to share commonalities). The algorithm then returns the best matching subsequence pair between the two.

How Smith-Waterman works:

To find common subsequences between two sequences, the Smith-Waterman algorithm follows three main steps:

1. **Initialization**

   Smith-Waterman uses a matrix to score the two sequences (scoring matrix). Example: find the common subsequence between the two genetic sequences "ATGCT" and "AGCT". The size of the scoring matrix is $(M + 1) \times (N + 1)$.

		0	1	2	3	4	5
			A	T	G	C	T
0							
1	A						
2	G						
3	C						
4	T						

Figure 1: *Initialization of the scoring matrix*

Initialize zeros on the rows: $\{(0,0), (0,1), (0,2), (0,3), (0,4), (0,5)\}$ and columns: $\{(0,0), (1,0)\ (2,0), (3,0), (4,0)\}$ that are closest to each character of the two sequences, including the cell at the top-left of the scoring matrix. The rows and columns are highlighted in green.

This initialization is formally defined as:

$$H(i, 0) = H(0, j) = 0$$

		0	1	2	3	4	5
			A	T	G	C	T
0		0	0	0	0	0	0
1	A	0					
2	G	0					
3	C	0					
4	T	0					

Figure 2: *Zero-initialization of the rows and columns*

2. Matrix-Filling

To compute the scores of the remaining cells. Start at the upper-leftmost cell with no value yet (highlighted in purple). The calculation of the value of the purple cell takes into account the neighboring cells (highlighted in green).

		0	1	2	3	4	5
			A	T	G	C	T
0		0	0	0	0	0	0
1	A	0					
2	G	0					
3	C	0					
4	T	0					

Figure 3: *Calculation of the remaining cells with the neighbors taken into account*

Focusing on the cell at $(1, 1)$, which is highlighted in purple. First, start with the cell to its left, which is $(1, 0)$, highlighted in red in *Figure 4*. For this example, $d = 2, m = \mu = 1$.

In comparison against $(1, 1)$:
$(1, 0) \rightarrow H(i, j - 1) - d = -2$
$(0, 1) \rightarrow H(i - 1, j) - d = -2$
$(0, 0) \rightarrow H(i - 1, j - 1) + s(a_i, b_j) \rightarrow match = 1$
$0 \rightarrow for\ reset$

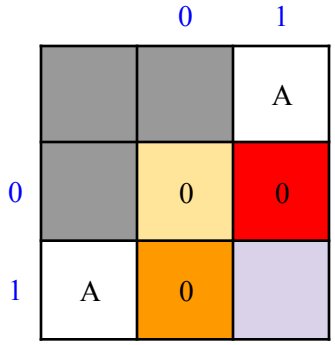


Figure 4: *Calculation of the remaining cells with the neighbors taken into account*

Since the recurrence relation uses a max function, the score in $(1, 1)$ is 1.

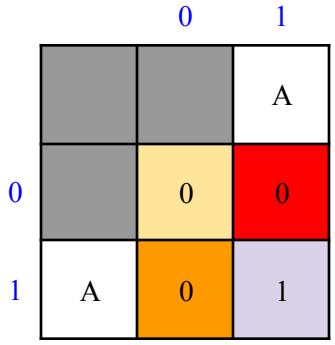


Figure 5: *Resulting score for* $(1, 1)$

Repeat the process by proceeding towards the cell in (1,2), repeat the neighbor calculation, until the entirety of cells in row 1 has a score, then proceed to row 2, and so on…

		0	1	2	3	4	5
			A	T	G	C	T
0		0	0	0	0	0	0
1	A	0	1				
2	G	0					
3	C	0					
4	T	0					

Figure 6: *Next iterations of the Matrix-Filling for Smith-Waterman*

When done, the resulting scoring matrix should be as follows:

		0	1	2	3	4	5
			A	T	G	C	T
0		0	0	0	0	0	0
1	A	0	1	0	0	0	0
2	G	0	0	0	1	0	0
3	C	0	0	0	0	2	0
4	T	0	0	0	0	0	3

Figure 7: *Completed scoring matrix*

3. **Traceback**

To find the locally aligned subsequence, simply find the values in the matrix that are greater than 1, and move up diagonally towards 0.

			0	1	2	3	4	5
			A	T	G	C	T
0		0	0	0	0	0	0
1	A	0	1	0	0	0	0
2	G	0	0	0	1	0	0
3	C	0	0	0	0	2	0
4	T	0	0	0	0	0	3

Figure 8: *Finding the optimal locally aligned subsequence*

Return the subsequence with the highest score. In this case, the subsequence to be returned is "GCT" since it has the highest score.

Recurrence relation:

The recurrence relation for the Smith-Waterman Algorithm is defined below, as well as the substitution score $s(a_i, b_j)$.

$$H(i,j) = max \begin{cases} 0, & \text{Reset} \\ H(i-1, j-1) + s(a_i, b_j), & Match\ /\ Mismatch \\ H(i-1,j) - d, & Gap\ in\ sequence\ b\ (Deletion) \\ H(i, j-1) - d & Gap\ in\ sequence\ a\ (Insertion) \end{cases}$$

$$with\ H(i,0) = H(0,j) = 0$$

Equation 1: *Smith-Waterman algorithm recurrence relation*

The Smith-Waterman algorithm is guaranteed to find the best local alignment of two sequences [4]. The recurrence in *Eq. 1* computes each matrix entry $H(i,j)$ as the maximum of four candidate scores: the diagonal neighbor extended by a match or mismatch, the upper and left neighbors extended by a gap, and zero. The zero acts as the floor value to ensure that the scoring matrix does not contain negative numbers. It serves as the "reset" in the algorithm to ensure that the cell does not inherit the bad alignment score from the previous calculation, effectively one of the key mechanisms that allows the algorithm to find the local alignment instead of the global alignment (by preventing the inheritance of the penalty from preceding cells).

$$s(a_i, b_j) = \begin{cases} +m, \text{ if } a_i = b_j \\ -\mu, \text{ if } a_i \neq b_j \end{cases} \qquad m, \mu > 0$$

Equation 2: *The substitution score (simple scheme)*

In *Eq. 2*, the substitution score $s(a_i, b_j)$ rewards a match with $+ m$ and penalizes a mismatch with $- \mu$.

Pseudocode:

```
ALGORITHM SmithWaterman(a[1..M], b[1..N], m, μ, d)

  // Step 1: Initialization
  for i ← 0 to M:  H[i][0] ← 0
  for j ← 0 to N:  H[0][j] ← 0
  maxScore ← 0
  (maxI, maxJ) ← (0, 0)

  // Step 2: Matrix filling
  for i ← 1 to M:
      for j ← 1 to N:
          if a[i] = b[j] then s ← +m else s ← −μ

          diag ← H[i-1][j-1] + s        // match / mismatch
          up   ← H[i-1][j]   − d        // gap in b (deletion)
          left ← H[i][j-1]   − d        // gap in a
(insertion)

          H[i][j] ← max(0, diag, up, left)

          if H[i][j] > maxScore then
              maxScore ← H[i][j]
              (maxI, maxJ) ← (i, j)

  // Step 3: Traceback
  alignA ← ""; alignB ← ""
  i ← maxI; j ← maxJ
  while i > 0 and j > 0 and H[i][j] > 0:
      if a[i] = b[j] then s ← +m else s ← −μ
      if H[i][j] = H[i-1][j-1] + s then       // diagonal
          prepend a[i] to alignA; prepend b[j] to alignB
          i ← i − 1; j ← j − 1
      else if H[i][j] = H[i-1][j] − d then    // up: gap in b
          prepend a[i] to alignA; prepend '-' to alignB
          i ← i − 1
      else                                    // left: gap in a
          prepend '-' to alignA; prepend b[j] to alignB
          j ← j − 1
```

```
    return alignA, alignB, maxScore,
           start = (i+1, j+1), end = (maxI, maxJ)
```

Figure 8: *Smith-Waterman algorithm pseudocode*

IV. Parallelization Approach

The project will implement the Smith-Waterman algorithm which uses two approaches: a sequential implementation in C and a parallel implementation in CUDA. The C implementation will serve as a baseline for evaluating the performance and correctness of the CUDA implementation. The algorithm is used to identify the most similar local regions between two sequences via construction of a dynamic programming scoring matrix. Each cell represents the best local alignment score for a portion of the input sequences and is computed via the Smith-Waterman recurrence relation formula (*see III. Proposed Algorithm*). The challenge in parallelizing this algorithm is that each cell depends on its upper, left, and upper-left neighbors. Thus, cells cannot be computed all at once. The CUDA implementation will process independent cells along the same anti-diagonal (top-right to bottom-left diagonal) while respecting dependencies between successive anti-diagonals. Both implementations will use the same scoring parameters and input sequences to ensure fair comparisons

Sequential C Implementation

The C implementation will construct the scoring matrix using nested loops, processing the cells row by row from top to bottom and left to right. For each cell, the program will calculate the possible diagonal, vertical, and horizontal scores, then select the maximum value while replacing negative scores with zero. The program will then track the highest score and its corresponding matrix during computation. After the scoring matrix is completed, the program will traceback to reconstruct the optimal local alignment, resulting in aligned sequences, gap positions, alignment score, and start and end positions, which will serve as reference results for the comparison with the CUDA implementation.

This implementation will also provide a baseline execution time used to calculate the speedup achieved through GPU parallelization since computations here are executed sequentially on the CPU.

CUDA Parallel Implementation

The CUDA implementation will use GPU threads to compute multiple scoring matrix cells concurrently. The advantage CUDA holds compared to C is the ability to calculate multiple cells at once, allowing it to exploit independent cells belonging to the same anti-diagonal. The implementation will consist of the following:

1. Memory allocation and data transfer. Allocate GPU memory for the input sequences and scoring matrix, then transfer the necessary input data from CPU to GPU
2. Wavefront computation. Divide the scoring matrix into anti-diagonals. For each anti-diagonal, assign CUDA threads to calculate valid cells independently using previously completed anti-diagonals
3. Synchronization. Ensure that all cells on the current anti-diagonal have been computed before moving to the next. The initial implementation will use successive kernel launches to have synchronization between anti-diagonals
4. Matrix completion. Anti-diagonal processing will continue until the entire scoring matrix

has been calculated. Track the maximum alignment score and its matrix coordinates

5. Traceback and alignment reconstruction. Transfer the completed scoring matrix or required traceback information back to the CPU. Perform traceback to reconstruct the optimal local alignment, including aligned sequences and gap positions

This approach preserves the data dependencies of the Smith–Waterman algorithm while allowing the GPU to process independent cells at the same time. If performance measurements show that kernel-launch overhead significantly limits speedup, a tiled wavefront approach may be recommended to be investigated for future optimization.

The focus of the CUDA parallelization is the construction of the parallelization matrix. The operations that will initially remain on the CPU are input processing (reading and validating input sequences and scoring parameters), final alignment processing (traceback and aligned sequence reconstruction), and output generation. Keeping traceback and output generation on the CPU would simplify the initial implementation and allow the project to concentrate on accelerating the intensive scoring matrix construction. Both versions will preserve the information needed to reconstruct and compare the full alignment.

CUDA Parallel Implementation

Both implementations will be tested using identical sequence pairs and scoring parameters. Correctness will be evaluated by comparing maximum alignment scores, alignment coordinates, reconstructed sequences, and gap placement. If multiple optimal alignments exist for the same input, both implementations may produce different aligned sequences while achieving the same optimal score. Therefore, the comparison will also verify that each reconstructed alignment is valid and achieves the expected score under the selected scoring scheme.

Performance will be evaluated using execution time and speedup:

$$Speedup = T_{sequentialC} / T_{CUDA}$$

The project will report the results of the scoring of the computation time and the end-to-end execution time, which will account for relevant CPU-GPU data transfers, GPU computation, and alignment reconstruction. This distinction will help determine whether CUDA provides a practical performance improvement after accounting for data-transfer and kernel-launch overhead. The final results will demonstrate the performance difference between sequential CPU execution and GPU-parallel execution while ensuring that both implementations produce valid local sequence alignments.

V. Implementation

The project will be developed mainly on Google Colab. It runs CUDA on a cloud GPU with no local setup, so every member can compile and run the same code even without an NVIDIA graphics card, usually a T4. However, the exact GPU depends on availability. Members who have an NVIDIA graphics card may also build and test locally with Visual Studio 2019/2022 and the CUDA toolkit. Final timing results will be taken on one platform (Google Colab) so that the sequential C and CUDA versions are compared on the same machine. Both versions will be written in C/C++. The sequential baseline will use standard C, while the parallel version will use CUDA C/C++ with the kernels launched from the host code.

The sequential C version will be implemented first. It reads the sequences and parameters, builds the scoring matrix row by row, tracks the highest score and its position, and

then performs traceback to rebuild the alignment. This version serves as the reference for both correctness and timing. The CUDA version will then be implemented. It allocates GPU memory for the sequences and the scoring matrix, then copies the inputs to the GPU. One kernel is launched per anti-diagonal so that each thread computes one cell, and the next launch only starts after the previous anti-diagonal is finished. The maximum score and its position are tracked during the computation. Once the matrix is complete, it is copied back to the CPU, where traceback rebuilds the alignment.

Testing will start with short sequences that can be checked by hand, and then move to longer sequences. The outputs of the two versions will be compared based on score, coordinates, and whether the alignment is valid, as described in Section IV.

For timing, each run will record the computation time (matrix filling only) and the end-to-end time (data transfer, GPU computation, and traceback). The C program will be timed with clock() or a similar high-resolution timer, and the CUDA program will be timed with CUDA events. Each test will be repeated several times and averaged. Nsight Systems will be used to check how much time goes to kernel launches and memory transfers. Sequence lengths will also be increased (for example, 1,000, 5,000, and 10,000 characters) to see how the speedup changes with input size. Larger inputs will be limited by GPU memory because the full matrix is stored for traceback.

The GitHub repository will contain the source code for both versions, sample input files, a README with build and run instructions, a discussion of the parallel algorithm, and the execution time comparison between sequential and parallel.

VI. Mid-Project Milestones

Week	Milestone	Deliverable
7	Finalize scoring parameters and test sequences. Set up the Colab notebook and GitHub repository	GitHub repository with README and sample input files
8	Implement the sequential C version (input reading, matrix filling, score tracking)	Working C program that outputs the score matrix and maximum score
9	Add traceback to the C version. Record baseline execution times	Verified C program with alignment output and the baseline timings
10	Mid-project demo	C baseline and working CUDA matrix filling
11	Add traceback and alignment reconstruction to CUDA version. Compare its output with the C version	CUDA program that outputs the full alignment with correctness comparison

12	Run performance tests on larger inputs. Profile with nsys. Compute speedup. If kernel-launch overhead is high, try the tiled wavefront approach.	Execution time tables, speedup results, and profiling notes
13	Finalize the documentation and prepare for the demo	Final submission and demo, Github repository

VII. References

[1] T. F. Smith and M. S. Waterman, “Identification of common molecular subsequences,” *J. Mol. Biol.*, vol. 147, no. 1, pp. 195–197, Mar. 1981, doi: 10.1016/0022-2836(81)90087-5. [Online]. Available: https://cs.brown.edu/courses/csci1810/fall-2024/resources/ch1_readings/Smith-Waterman-paper%202.pdf

[2] M. Farrar, “Striped Smith–Waterman speeds database searches six times over other SIMD implementations,” *Bioinformatics*, vol. 23, no. 2, pp. 156–161, Jan. 2007, doi: 10.1093/bioinformatics/btl582.

[3] A. Khajeh-Saeed, S. Poole, and J. B. Perot, “Acceleration of the Smith–Waterman algorithm using single and multiple graphics processors,” *J. Comput. Phys.*, vol. 229, no. 11, pp. 4247–4258, Jun. 2010, doi: 10.1016/j.jcp.2010.02.009.

[4] A. Coghlan, “The Smith-Waterman algorithm,” Wellcome Trust Sanger Institute, SlideShare. [Online]. Available: https://www.slideshare.net/slideshow/the-smith-waterman-algorithm/16572305 (accessed Oct. 24, 2026).

[5] B. Schmidt, F. Kallenborn, A. Chacon, and C. Hundt, “CUDASW++4.0: Ultra-fast GPU-based Smith–Waterman protein sequence database search,” *BMC Bioinformatics*, vol. 25, Art. no. 342, Nov. 2024, doi: 10.1186/s12859-024-05965-6.